CFG NINJA

SECURITY ASSESSMENT

LOOP Protocol Principal withdrawal mechanism...

October 25, 2026

Limited-Scope Review (no score issued)



BLADE POOL

RISK ANALYSIS | LOOP Protocol.

Classifications of Manual Risk Results

Note: Risk classifications follow CVSS v4.0 standards (https://www.first.org/cvss/v4.0/specification-document). Contract security metrics are verified through GoPlus API token scanner technology, which provides real-time validation of contract parameters including

Classification	CVSS Range	Description
Critical	9.0 - 10.0	Critical vulnerabilities that pose immediate and severe risks requiring urgent attention.
High	7.0 - 8.9	High-priority issues that could lead to significant security breaches or financial loss.
Medium	4.0 - 6.9	Medium-severity findings that should be addressed to improve contract security.
Low	0.1 - 3.9	Low-risk items or best practice suggestions with minimal security impact.
Informational	0.0	Informational findings about detected functions or contract features.

Manual Code Review Risk Results

Contract Security	Description
Buy Tax	N/A
Sale Tax	N/A
Cannot Buy	N/A
Cannot Sale	N/A
Max Tax	N/A
Modify Tax	N/A
Fee Check	Not Applicable
Is Honeypot?	Not Applicable
Trading Cooldown	Not Applicable
Enable Trade?	N/A

Contract Security	Description
Pause Transfer?	N/A
Max Tx?	N/A
Is Anti Whale?	N/A
Is Anti Bot?	N/A
Is Blacklist?	N/A
Blacklist Check	N/A
Is Whitelist?	N/A
Can Mint?	Not applicable - program review, no mint authority in this program
Is Proxy?	N/A
Can Take Ownership?	Program upgrade authority not evidenced on-chain - see LOOP-01 and LOOP-14
Hidden Owner?	N/A
Owner	Program upgrade authority (verify on-chain)
Self Destruct?	N/A
External Call?	Not assessed in this report. The audited path performs SPL token transfers only; the program's cross-program invocations into Raydium, the loop_contract fee sweep and the referral program are outside this scope.
Other?	Solana Anchor staking and referral program, anchor-lang 1.1.2. This report audits one mechanism, the principal custody and withdrawal path in instructions/stake.rs, on the source supplied 24 October 2026. The external loop_contract and referral programs carry protocol logic and have never been supplied.
Holders	N/A
Audit Confidence	Limited-scope (principal withdrawal mechanism) review - no score issued

Contract Security	Description
Authority Check	Admin and operator authorization verified correct in source; upgrade authority not evidenced on-chain
Freeze Check	N/A

Overview of identified vulnerabilities and risks. See the full report for detailed methodology and recommendations.

CFG Ninja Verified on October 25, 2026

LOOP Protocol

Executive Summary

TYPES	ECOSYSTEM	LANGUAGE
DeFi	Solana	Rust

Timeline

Audit Request	Onboarding Process	Audit Preview	Audit Release
2026-09-15	2026-09-15	2026-10-25	2026-10-25

Vulnerability Summary



1	0	1	1
Total Findings	Resolved	Pending	Unresolved

Limited-scope review: only the principal custody and withdrawal path in instructions/stake.rs was audited. One Critical finding is recorded below. It is acknowledged by the project as intentional by design and is NOT remediated, so it is counted as open. No security score and no whole-program pass or fail status is issued.

Severity	Resolution Status	Description
1 Critical	0 Resolved, 1 Pending	Immediate danger. Can lead to loss of user funds, unauthorized access, or full system compromise.
0 High		Significant risk that should be addressed urgently to prevent exploitation or financial loss.
0 Medium		Moderate risk that can lead to issues if combined with other weaknesses; should be addressed.
0 Low		Minor concern or best-practice deviation with limited direct security impact.
0 Informational		Not a vulnerability - code-quality, documentation, or optimization suggestions.

PROJECT OVERVIEW | LOOP Protocol.

Token Summary

Parameter	Result
Address	81EUhaSVy8edRdyZ2TSrZSYa72U2tKY8zTDURsUAtRT6
Name	LOOP Protocol
Token Tracker	LOOP Protocol (LOOP)
Decimals	N/A
Supply	N/A (program)
Platform	Solana
Compiler	Anchor (Rust / Solana BPF)
Contract Name	staking (program)
Optimization	No
LicenseType	See source header
Language	Rust
Codebase	https://x.com/loop_protocol_

Audited Files

File	SHA-1
instructions/stake.rs	7742005E3C95D34897F1E3D714CA5F7B2402CF3C

Audited file: instructions/stake.rs, 17,011 bytes, 572 lines, of which only the principal custody and withdrawal path is in scope: the transfer of principal into the vault on a new stake, the reinvest branch that returns it, and the conditions that gate that branch. Reward accrual, referral settlement, tier inventory, fee pricing and every other aspect of this file are NOT in scope. The following modules were read as supporting references, so that the absence of any other withdrawal path could be established rather than assumed, and are not themselves in scope: lib.rs (SHA-1 4F2B1B72A5D58129AA2AFAC9FB8AEAA306732B17), instructions.rs (SHA-1 EC33B5EBE08AD1339319BC1CE4ECC9F0D1444FBC), instructions/admin.rs (SHA-1 FF00C3EC4573A7215F8C05DB18B66F4EFB9ECFE0), instructions/claim_all.rs (SHA-1 DD3D65BE43D688FDAE135F31B8FAFA2D1B8E6947), instructions/claim_static.rs (SHA-1 25583B0FABBE9E897DE329A8E0D94BCD1B9CDE8A), instructions/claim_dynamic.rs (SHA-1 5889BD8E646BF6D94170856EB67943E8AAD33258), instructions/claim_peer.rs (SHA-1 C63E0D4FC2FCAA5E46E2A63788B4D5CA9D7805B0), state.rs (SHA-1 7A2968085BADF21A798953B847ADE6782FB6328B). All hashes are computed from the files as supplied by the project on 24 October 2026, so anyone holding the same files can confirm this report describes them and not another revision.

TECHNICAL FINDINGS

Smart contract security audits classify risks into several categories: Critical, High, Medium, Low, and Informational. These classifications help assess the severity and potential impact of vulnerabilities found in smart contracts.

Classification of Risk

Severity	CVSS Range	Description
Critical	9.0 - 10.0	Immediate danger. Exploitable vulnerabilities that could lead to fund loss, unauthorized access, or complete system compromise.
High	7.0 - 8.9	Significant security risk. Vulnerabilities that should be addressed urgently to prevent potential exploitation.
Medium	4.0 - 6.9	Moderate security risk. Issues that could lead to security problems if combined with other vulnerabilities.
Low	0.1 - 3.9	Minor security concern. Best practice violations or low-impact issues.
Informational	0.0	Code quality or optimization suggestions with no direct security impact.

Detailed Security Findings

Each finding lists its severity, an indicative CVSS, the source location, a description, our recommendation and its current status, with the affected source excerpt and a suggested fix where the remediation is code-expressible. Findings apply to the audited scope only.

LOOP-24 | Staked Principal Cannot Be Withdrawn From The Program By Any Party.

Category	Severity	CVSS	Location	Status
Access to Funds	Critical	7.5	instructions/stake.rs: handle_stake (reinvest branch); program-wide absence of a withdrawal instruction	Acknowledged - Intentional by Design (not remediated)

Description

In the source supplied on 24 October 2026 the claim_principal instruction has been removed: the handler file is gone, the module is no longer declared in instructions.rs, and the entry point is no longer exposed by the program router in lib.rs. It was the only instruction by which a user could withdraw staked principal. We established the consequence by enumerating every transfer whose source is the principal vault rather than by inference. There are nine such transfers across the program. Eight are in claim_static, claim_dynamic, claim_peer and claim_all, and every one of them moves accrued static, dynamic or peer rewards, or the project fee split, and never touches Order.principal. The ninth, in handle_stake, is the only transfer of principal back to a user, and it is unreachable except inside the reinvest branch. That branch does not constitute a withdrawal. Entering it requires calling stake again, and prepare_stake enforces that the new principal equals required_reinvest_principal for the maturing order, which is the same amount, or a larger one when the order is capped. The matching inbound transfer later in the same instruction moves that amount straight back into the vault. The user's principal is therefore returned and re-locked within a single transaction, the user nets nothing, and the capital is committed for a further term. No administrative path exists either. The entire administrative surface is set_min_stake and fund_reward_vault, and fund_reward_vault transfers into the vault, not out of it. There is no emergency withdrawal, no pause-and-return, and no sweep. The result is that once principal is staked, neither the user nor the admin can recover it. At maturity the only action the program permits is to stake again. The project confirmed on 25 October 2026 that this is intentional and consistent with their business logic, and accepted its classification as a Critical finding. It is rated on impact, as this methodology rates every finding: the impact is the permanent and total loss of access to deposited principal, for every order and every user, and intent does not reduce it. The indicative CVSS vector describes availability impact with no attacker, because there is no attack here. This is a property of the design, not an exploitable condition, and the CVSS figure should be read as an indication of severity rather than as a description of a technique. One limitation bounds what this report can say. No deployed program address or upgrade authority has been supplied in any round of this engagement, so we cannot state whether the deployed program is upgradeable. If it is, the team could add a withdrawal instruction later. That is a deployment property we have not been able to verify and it is not a property of this source.

Recommendation

If the lock is the intended product, the code is not the thing to change, but the disclosure is. Any user must be told before they deposit, in the staking interface itself and not only in documentation, that principal is not withdrawable and that the only action available at maturity is to stake again. The wording should be at least as plain as the wording used in this finding. A user who deposits believing they can exit at maturity has been misled by the absence of the instruction rather than by anything the program says. If an exit is wanted later, it requires a

new instruction, restoring the matured-order and already-claimed guards the removed handler carried, together with the upline bookkeeping it performed when a user became inactive. On a live deployment that requires the program upgrade authority, which has never been evidenced to us. Independently of either choice, publish the program's deployed address, its current upgrade authority and whether the admin key is a multisig. With no withdrawal path in the program, the upgrade authority is the only thing that could ever create one, which makes who holds it a material fact for any user of this protocol.

Mitigation

Acknowledged by the project as intentional by design, and NOT remediated. The project confirmed in writing on 25 October 2026 that the inability to withdraw principal independently is deliberate and consistent with their platform's business logic, and raised no objection to this finding being recorded at Critical severity. No code change has been made and none is expected. The finding is open and is counted as open in the vulnerability summary of this report. Acknowledgement by the project is not remediation and this report does not treat it as such.

References:

https://www.anchor-lang.com/docs/account-constraints
https://github.com/coral-xyz/sealevel-attacks
https://solana.com/developers/courses/program-security

Affected Code

```
// instructions/stake.rs - the ONLY transfer of principal back to a user,
// reachable only when an order is maturing and is being reinvested
if let Some(idx) = reinvest_index {
    ...
    if !already_claimed {
        transfer_checked(
            CpiContext::new_with_signer(
                ctx.accounts.neverzero_token_program.key(),
                TransferChecked {
                    from: ctx.accounts.vault_neverzero_account.to_account_info(),
                    to:   ctx.accounts.user_neverzero_account.to_account_info(),
                    ...
                },
                &[seeds],
            ),
            old_principal,   // <- returned to the user ...
            ctx.accounts.neverzero_mint.decimals,
        )?;
    }
}

transfer_checked(
    ...
        from: ctx.accounts.user_neverzero_account.to_account_info(),
        to:   ctx.accounts.vault_neverzero_account.to_account_info(),
    ...
    principal,               // <- ... and taken straight back in
)?;

// and in prepare_stake, which is what makes the two cancel out:
let required = required_reinvest_principal(&stake_user.orders[idx])?;
require!(principal == required, ErrorCode::InvalidReinvestAmount);
```

Suggested Fix

```
// Only if an exit is wanted. This restores the guards the removed
// claim_principal handler carried; on a live deployment it needs the
// program upgrade authority.
pub fn handle_withdraw_principal(
    ctx: Context<WithdrawPrincipal>,
    order_id: u64,
) -> Result<()> {
    let now_ts = Clock::get()?.unix_timestamp;
    let principal = {
        let mut stake_user = ctx.accounts.stake_user.load_mut()?;
        let order = stake_user.find_order_mut(order_id)?;
        require!(!is_set(order.principal_claimed), ErrorCode::PrincipalAlreadyClaimed);
        require!(now_ts >= order.maturity_day, ErrorCode::OrderNotMatured);
        order.principal_claimed = flag(true);
        order.principal
    };
    require!(
        ctx.accounts.vault_neverzero_account.amount >= principal,
        ErrorCode::InsufficientVault
    );
    // transfer vault -> user, signed by the config PDA, then
    // sync_active_from_orders and refresh the upline bookkeeping.
    Ok(())
}
```

FINDINGS

In this document, we present the findings and results of the smart contract security audit. The identified vulnerabilities, weaknesses, and potential risks are outlined, along with recommendations for mitigating these issues. It is crucial for the team to address these findings promptly to enhance the security and trustworthiness of the smart contract code.

Severity	Found	Pending	Resolved
Critical	1	1	0
High	0	0	0
Medium	0	0	0
Low	0	0	0
Informational	0	0	0
Total	1	1	0

In a smart contract, a technical finding summary refers to a compilation of identified issues or vulnerabilities discovered during a security audit. These findings can range from coding errors and logical flaws to potential security risks. It is crucial for the project owner to thoroughly review each identified item and take necessary actions to resolve them. By carefully examining the technical finding summary, the project owner can gain insights into the weaknesses or potential threats present in the smart contract. They should prioritize addressing these issues promptly to mitigate any risks associated with the contract's security. Neglecting to address any identified item in the security audit can expose the smart contract to significant risks. Unresolved vulnerabilities can be exploited by malicious actors, potentially leading to financial losses, data breaches, or other detrimental consequences. To ensure the integrity and security of the smart contract, the project owner should engage in a comprehensive review process. This involves understanding the nature and severity of each identified item, consulting with experts if needed, and implementing appropriate fixes or enhancements. Regularly updating and maintaining the smart contract's codebase is also essential to address any emerging security concerns. By diligently reviewing and resolving all identified items in the technical finding summary, the project owner can significantly reduce the risks associated with the smart contract and enhance its overall security posture.

SOCIAL MEDIA CHECKS | LOOP Protocol.

Social Media	URL	Result
Website	https://x.com/loop_protocol_	Confirmed
Telegram		N/A
Twitter	https://x.com/loop_protocol_	Confirmed
Facebook		N/A
Reddit	N/A	N/A
Instagram	N/A	N/A
CoinGecko		N/A
Github	N/A	N/A
CMC		N/A
Email		Contact
Other		N/A

From a security assessment standpoint, inspecting a project's social media presence is essential. It enables the evaluation of the project's reputation, credibility, and trustworthiness within the community. By analyzing the content shared, engagement levels, and the response to any security-related incidents, one can assess the project's commitment to security practices and its ability to handle potential threats.

Social Media Information Notes:

Auditor Notes: Project X/Twitter (x.com/loop_protocol_) provided by the client.

Project Owner Notes: Partial-scope engagement.

Assessment Results | Final Audit Score LOOP.

Review	Score
Reviewed scope	instructions/stake.rs - principal custody and withdrawal path only
Out of scope for this report	reward accrual, referral settlement, tier inventory and fee pricing in the same file; every other module of the program; and the external loop_contract and referral programs
Security score	Not issued (limited-scope review)

This report reviews one mechanism and nothing else: how staked principal enters the program and whether it can leave. Within instructions/stake.rs the principal custody and withdrawal path was reviewed line by line, together with the conditions that gate the only branch that returns principal. The modules named as references on the Audited Files page were read so that the absence of any other withdrawal path could be established by enumerating every transfer out of the principal vault, rather than assumed. OUT OF SCOPE: reward accrual, referral settlement, tier inventory and fee pricing within the same file, every other module of the staking program, and the external loop_contract and referral programs, which have never been provided. No security score and no pass or fail status is issued in this edition. The reason is stated plainly: this methodology's weighted table deducts ten points for a Critical finding, so a report whose only finding is a single Critical would carry a headline in the nineties. For a report about principal that cannot be withdrawn, that number would misrepresent the finding it sits above, so none is issued. Two further limitations apply. No deployed program address, upgrade authority or administrative key has been supplied in any round of this engagement, and the submission does not build as supplied, so no statement in this report is tied to a program running on chain. And other modules of this program carry open findings, recorded in separate reviews by CFG Ninja. A limited-scope report on one mechanism is not a clean bill of health for LOOP Protocol, and no whole-program pass or fail is issued here.

LIMITED-SCOPE REVIEW - NO SCORE ISSUED

This report covers the principal withdrawal mechanism in instructions/stake.rs only and is not a full-program audit. One Critical finding is recorded, acknowledged by the project as intentional by design and not remediated. Other modules carry open findings recorded in separate reviews.

Important Notes for LOOP

Scope and Status

PRINCIPAL WITHDRAWAL MECHANISM REVIEW - 2026-10-25 - LIMITED SCOPE, NO SCORE ISSUED

This report reviews one mechanism and nothing else: how staked principal enters LOOP Protocol and whether, and by what path, it can leave. The project requested this scope on 25 October 2026. The audited file is instructions/stake.rs, and within that file only the principal custody and withdrawal path is in scope. It is not a full-program audit. Other modules of this program carry open findings, recorded in separate reviews by CFG Ninja.

One finding is recorded, at Critical severity. The project has confirmed that the condition it describes is intentional, and the finding is recorded as acknowledged and not remediated. It is counted as open in the vulnerability summary.

Why this report issues no score

This methodology scores by weighted deduction from a starting figure of 100, in which a Critical finding deducts ten points. Applied mechanically to a report whose only finding is a single Critical, that table would return a figure in the nineties once the project's verified public presence is added. Printed above a finding which states that staked principal cannot be recovered by anyone, a number of that kind would misrepresent the report to any reader who saw only the headline.

So none is issued. There is no security score and no pass or fail status in this edition. The finding page, the vulnerability summary and this scope statement are the whole of what this report asserts.

What was audited

The Audited Files table in the Project Overview records the SHA-1 of the file this report covers, so the scope can be verified against the file the project supplied rather than taken on trust. The hashes are computed from the source supplied on 24 October 2026. In that revision the project renamed A9 to Neverzero and A8 to Loop, and this report uses those names because that is what the code says.

instructions/stake.rs, 572 lines, of which the principal custody and withdrawal path is in scope: the transfer of principal into the vault on a new stake, the reinvest branch that returns it, and the conditions in prepare_stake that gate that branch.

```
Read as supporting references, and not themselves in scope: lib.rs and instructions.rs, for which entry points the program exposes and which modules it declares; admin.rs, for the whole of the administrative surface; claim_all.rs, claim_static.rs, claim_dynamic.rs and claim_peer.rs, for what each reward path moves; and state.rs, for the fields that record an order's principal and whether it has been claimed.
```

Out of scope for this report

Reward accrual, referral settlement, tier inventory and fee pricing within the same file. Every other module of the staking program. The external loop_contract and referral programs, which have never been provided.

```
Two further limitations apply. No deployed program address, upgrade authority or administrative key has been supplied in any round of this engagement, and the submission does not build as supplied, so no statement in this report is tied to a program running on chain. It describes the source as supplied, not the program users interact with. And a limited-scope report on one mechanism is not a clean bill of health for LOOP Protocol; no whole-program pass or fail is issued here.
```

How the conclusion was established

The central claim of this report is a negative one, that no withdrawal path exists, and a negative claim cannot rest on not having noticed something. It was established by enumeration instead.

Every transfer in the program whose source is the principal vault was listed. There are eight. Seven are in the four reward handlers, at claim_all.rs lines 269 and 286, claim_dynamic.rs lines 172 and 189, claim_peer.rs lines 172 and 189, and claim_static.rs line 213. Every one of them moves accrued static, dynamic or peer rewards, or the project fee split, and none of them reads Order.principal. The eighth, at stake.rs line 516, is the only transfer of principal back to a user.

The administrative surface was then enumerated in full. It consists of set_min_stake and fund_reward_vault, and fund_reward_vault transfers into the vault, not out of it. There is no emergency withdrawal, no pause-and-return and no sweep.

Finally the router was checked against the module list. claim_principal, which in the previous revision was

the instruction by which a user withdrew principal, is no longer declared in instructions.rs, no longer exposed in lib.rs, and its handler file is no longer present in the source.

The finding

LOOP-24, Critical, acknowledged as intentional by design and not remediated.

The single transfer of principal back to a user sits inside the reinvest branch of handle_stake, and that branch is not a withdrawal. Reaching it requires calling stake again. prepare_stake enforces that the new principal equals required_reinvest_principal for the maturing order, which is the same amount, or a larger one when the order is capped. The matching inbound transfer later in the same instruction moves that amount straight back into the vault. Principal is therefore returned and re-locked within a single transaction, the user nets nothing, and the capital is committed for a further term.

The consequence is that once principal is staked, neither the user nor the administrator can recover it. At maturity the only action the program permits is to stake again.

The project confirmed in writing on 25 October 2026 that this is intentional and consistent with its platform's business logic, and raised no objection to the finding being recorded at Critical severity.

The rating is on impact, as every rating in this methodology is. The impact is the permanent and total loss of access to deposited principal, for every order and every user, and intent does not reduce it. The CVSS figure shown on the finding page is indicative only: it describes an availability impact with no attacker, because there is no attack here. This is a property of the design rather than an exploitable condition.

One mitigating possibility cannot be assessed. If the deployed program is upgradeable, the team could add a withdrawal instruction later. No deployed address or upgrade authority has been supplied in any round of this engagement, so whether that is possible is unknown to us, and it is not a property of the source reviewed here.

What acknowledged means in this report

It means the project has been shown the finding, agrees the condition exists, and states it is deliberate. It does not mean the finding is fixed, and this report does not treat it as fixed. The vulnerability summary counts it as open. No remediation has been made and none is expected.

What this report does and does not mean

It establishes, by enumeration rather than by inference, that in the source supplied on 24 October 2026 there is no instruction through which staked principal can be withdrawn from LOOP Protocol, by a user or by an administrator, and that the single path which returns principal re-locks it in the same transaction.

It does not assess anything else. It says nothing about reward accrual, referral settlement, fee pricing, account validation or the external programs this one calls into. It does not tie any statement to a program running on chain. It is not a full-program audit, other modules carry open findings recorded in separate reviews, and no score and no pass or fail status is issued.

Appendix

Finding Categories

Centralization / Privilege

Centralization / Privilege findings refer to either feature logic or implementation of components that act against the nature of decentralization, such as explicit ownership or specialized access roles in combination with a mechanism to relocate funds.

Gas Optimization

Gas Optimization findings do not affect the functionality of the code but generate different, more optimal EVM opcodes resulting in a reduction in the total gas cost of a transaction.

Logical Issue

Logical Issue findings detail a fault in the logic of the linked code, such as an incorrect notion of how block.timestamp works.

Control Flow

Control Flow findings concern the access control imposed on functions, such as owner-only functions being invokable by anyone under certain circumstances.

Volatile Code

Volatile Code findings refer to segments of code that behave unexpectedly in certain edge cases that may result in a vulnerability.

Coding Style

Coding Style findings usually do not affect the generated byte-code but rather comment on how to make the codebase more legible and, as a result, easily maintainable.

Inconsistency

Inconsistency findings refer to functions that should seemingly behave similarly yet contain different code, such as a constructor assignment imposing different require statements on the input variables than a setter function.

Coding Best Practices

ERC 20 Coding Standards are a set of rules that each developer should follow to ensure the code meets a set of criteria and is readable by all developers.

Disclaimer

Scope. This report documents the results of a security assessment and smart contract audit performed by CFG NINJA (the "Auditor") for the project specified in this report. The assessment covers only the deliverables and code expressly listed in the engagement; any changes, integrations, or subsequent deployments are outside the scope.

No Warranty. The Auditor performs the assessment using industry-standard practices but makes no representations or warranties, express or implied, including any warranty of merchantability, fitness for a particular purpose, or non-infringement. The Auditor does not guarantee that the audited code is free of bugs, vulnerabilities, or exploitable issues.

Limitation of Liability. To the maximum extent permitted by law, the Auditor and its affiliates, officers, employees, agents, and contractors shall not be liable for any indirect, incidental, special, consequential, or punitive damages, or for any loss of profits, revenue, data, or business opportunities, arising out of or related to the assessment, even if advised of the possibility of such damages. The Auditor's aggregate liability for direct damages is limited to the fees paid for the audit engagement.

Client Responsibility. The project owner/client retains sole responsibility for all decisions and actions taken in connection with the audited code, including fixing identified issues, deploying code to any environment, and applying security mitigations. The Auditor's recommendations are advisory; implementation and verification of fixes are the client's responsibility.

Third-Party Tools and Services. The Auditor may use third-party tools, services, or automated scanners as part of the assessment. Use of such tools is without warranty; the Auditor is not responsible for defects, failures, or inaccuracies arising from third-party services.

Confidentiality and Data Security. The Auditor will treat non-public client information as confidential. However, the Auditor cannot guarantee absolute security for data transmitted over the internet or third-party platforms. Client should take reasonable precautions to protect sensitive information.

Not Financial, Legal, or Investment Advice. The assessment is a technical security review only and is not financial, legal, tax, or investment advice. Clients should consult appropriate professionals for those matters.

Ownership and Governance Notes. If ownership is transferred, held by a multisig, or controlled by a governance contract, the client should document and disclose those arrangements; the Auditor's report reflects the ownership state observed during the engagement but may not reflect subsequent changes.

Governing Law and Counsel. This engagement and any disputes arising from it shall be governed by the laws agreed in the engagement terms. Clients are advised to seek legal counsel before relying on or publishing the report.

Acceptance. By proceeding with this engagement or using this report, the client acknowledges and accepts these terms.

